קטע קוד A

```
 1. using VarsMap = std::map<std::string, double>;
 2.
 3. struct BaseExpression {
 4.   virtual ~BaseExpression() {}
 5.   virtual double eval(const VarsMap* pvars = nullptr) const = 0;
 6.   virtual ostream& print(ostream& out) const = 0;
 7.   friend ostream& operator<<(ostream& o, const BaseExpression& e) {
 8.     return e.print(o);
 9.   }
10. };
11.
12. class Expression {
13. protected:
14.   const BaseExpression* const e;
15.   class PToken {}; // accessible only by protected classes
16. public:
17.   Expression(BaseExpression* e, PToken): e(e) {}
18.   Expression(const Expression& e) = delete;
19.   ~Expression() {
20.       cout << "about to delete " << *this << endl;
21.       delete e;
22.   }
23.   friend ostream& operator<<(ostream& out, const Expression& e) {
24.     return out << *e.e;
25.   }
26. };
27.
28. class RootExpression: public Expression {
29.   VarsMap vars;
30. public:
31.   RootExpression(BaseExpression* e): Expression(e, PToken{}) {}
32.   void setVar(const std::string& var_name, double value) {
33.       vars[var_name] = value;
34.   }
35.   void printVars() const {
36.     for(const auto& var: vars) {
37.         cout << var.first << " = " << var.second << std::endl;
38.     }
39.   }
40.   double eval() const { return e->eval(&vars); }
41. };
42.
43. struct InnerExpression: public Expression {
44.   InnerExpression(BaseExpression* e) : Expression(e, PToken{}) {}
45.   double eval(const VarsMap* pvars = nullptr) const {
46.     return e->eval(pvars);
47.   }
48. };
```

```
49. class Number: public BaseExpression {
50.   double value;
51. public:
52.   Number(double d) : value(d) {}
53.   double eval(const VarsMap* pvars = nullptr) const override {
54.     return value;
55.   }
56.   ostream& print(ostream& out) const override {
57.     return out << value;
58.   }
59. };
60.
61. class Var: public BaseExpression {
62.   const std::string name;
63. public:
64.   Var(const std::string& name) : name(name) {}
65.   double eval(const VarsMap* pvars = nullptr) const override {
66.       if(!pvars) return 0;
67.       auto itr = pvars->find(name);
68.       if(itr == pvars->end()) return 0;
69.       return itr->second;
70.   }
71.   ostream& print(ostream& out) const override {
72.     return out << name;
73.   }
74. };
75.
76. class BinaryExpression : public BaseExpression {
77.   InnerExpression e1, e2;
78.   virtual double evalImpl(double d1, double d2) const = 0;
79.   virtual char sign() const = 0;
80.   Number* to_expression(double num) const {
81.       return new Number(num);
82.   }
83.   Var* to_expression(const std::string& s) const {
84.       return new Var(s);
85.   }
86.   BaseExpression* to_expression(BaseExpression* e) { return e; }
87. public:
88.   template<typename E1, typename E2>
89.   BinaryExpression (E1 param1, E2 param2)
90.       : e1(to_expression(param1)), e2(to_expression(param2)) {}
91.   double eval(const VarsMap* pvars = nullptr) const override {
92.     return evalImpl(e1.eval(pvars), e2.eval(pvars));
93.   }
94.   ostream& print(ostream& out) const override {
95.     return out << '(' << e1 << ' ' << sign() << ' ' << e2 << ')';
96.   }
97. };
98.
```

```
class Sum: public BinaryExpression {
  double evalImpl(double d1, double d2) const override {
    return d1 + d2;
  }
  char sign() const override { return '+'; }
public:
  using BinaryExpression::BinaryExpression; // inheriting base ctor
};

class Mult: public BinaryExpression {
  double evalImpl(double d1, double d2) const override {
    return d1 * d2;
  }
  char sign() const override { return '*'; }
public:
  using BinaryExpression::BinaryExpression; // inheriting base ctor
};

int main() {
  auto e1 = new Sum( "x", 3 );
  RootExpression e2 = new Mult( e1, "x" );
  e2.setVar("x", 10);
  // e2.setVar("y", 100);
  e2.printVars();
  // std::cout << *e1 << " = " << e1->eval() << std::endl;
  std::cout << e2 << " = " << e2.eval() << std::endl;
  e2.setVar("x", 20);
  std::cout << e2 << " = " << e2.eval() << std::endl;
}
```

קטע קוד B

```
class BoolArray {
  char* arr = nullptr;
  size_t size = 0;
  size_t capacity = 0;

  static size_t actual_size(size_t num_bools) {
    if(num_bools == 0) return 0;
    return (num_bools - 1)/8 + 1;
  }
  void resize(size_t new_size) {
    if(new_size <= capacity) {
      size = new_size; return;
    }
    // allocate and initialize all to false
    char* new_arr = new char[actual_size(new_size)] {};
    // memcpy signature: std::memcpy(dst, src, size_in_bytes)
    std::memcpy(new_arr, arr, actual_size(size));
    delete []arr;
    size = new_size;
    capacity = actual_size(new_size) * 8;
    arr = new_arr;
  }
  class BoolProxy {
    BoolArray* const arr;
    size_t index;
    size_t byte_index() const { return index / 8; }
    size_t bit_index() const { return index % 8; }
    char bit_val() const { return 1 << bit_index(); }
  public:
    BoolProxy(BoolArray* arr, size_t index)
        : arr(arr), index(index) {}
    operator bool() const {
      if(index >= arr->size) return false;
      return arr->arr[byte_index()] & bit_val();
    }
    bool operator=(const BoolProxy& bp) {
      return *this = static_cast<bool>(bp);
    }
    bool operator=(bool value) {
      if(index >= arr->size) arr->resize(index + 1);
      if(value)
        arr->arr[byte_index()] |= bit_val();
      else
        arr->arr[byte_index()] &= ~(bit_val());
      return value;
    }
  };
```

```
public:
  BoolArray() {}
  ~BoolArray() { delete []arr; }
  BoolArray(const BoolArray&) = delete;
  BoolArray& operator=(const BoolArray&) = delete;
  bool operator[](size_t index) const {
    return const_cast<BoolArray&>(*this)[index];
  }
  BoolProxy operator[](size_t index) {
    return BoolProxy{this, index};
  }
  auto begin() { return arr; }
  auto end() { return arr + actual_size(size); }
};
```

להלן מספר main אפשריים:

B1 main

```
int main() {
  BoolArray barr1;
  for(size_t i = 0; i < 10; ++i) {
    barr1[i] = true;
  }
  barr1[12] = true;
}
```

B2 main

```
int main() {
   BoolArray barr2;
   barr2[5] = barr2[1];
   barr2[6] = !barr2[0];
   for(auto b : barr2) {
     std::cout << b << std::endl;
   }
}
```

B3 main

```
int main() {
  BoolArray barr3;
  std::cout << barr3[6] << std::endl;
  // bool val = true;
  // for(auto& b : barr3) {
  //   b = val;
  //   val = !val;
  // }
  for(auto b : barr3) {
    std::cout << b << std::endl;
  }
}
```

קטע קוד C – לשאלות הבונוס

```
 1. struct Expression {
 2.   virtual ~Expression() {}
 3.   virtual double eval() const = 0;
 4.   virtual ostream& print(ostream& out) const = 0;
 5.   virtual Expression* clone() && = 0;
 6.   friend ostream& operator<<(ostream& out, const Expression& e) {
 7.     return e.print(out);
 8.   }
 9. };
10.
11. template<typename ActualType>
12. class MultiExpression : public Expression {
13.
14.   std::vector<Expression*> subexpressions;
15.
16.   virtual double evalImpl(double d1, double d2) const = 0;
17.   MultiExpression (std::vector<Expression*>&& vec)
18.     : subexpressions(std::move(vec)) {}
19.   void free() {
20.     for_each(subexpressions.begin(),subexpressions.end(),
21.              [](Expression* e) {delete e;} );
22.   }
23.
24. public:
25.   template<typename... Ts>
26.   MultiExpression (Ts&&... e)
27.     : subexpressions({std::forward<Ts>(e).clone()...}) {}
28.   ~MultiExpression() override { free(); }
29.   MultiExpression(MultiExpression&& other)
30.     : MultiExpression(std::move(other.subexpressions)) {}
31.   MultiExpression& operator=(MultiExpression&& other) {
32.     free();
33.     subexpressions = std::move(other.subexpressions);
34.     // initialize other.subexpressions to an empty vector
35.     other.subexpressions = {};
36.     return *this;
37.   }
38.   MultiExpression* clone() && override {
39.     return new ActualType(std::move(subexpressions));
40.   }
41.   double eval() const override {
42.     double head = subexpressions[0]->eval();
43.     for_each(subexpressions.begin() + 1, subexpressions.end(),
44.         [&head, this](Expression* e) {
45.                 head = evalImpl(head, e->eval()); } );
46.     return head;
47.   }
48.
```

```
49.   ostream& print(ostream& out) const override {
50.     out << '(' << *subexpressions[0];
51.     for_each(subexpressions.begin() + 1,subexpressions.end(),
52.         [&out, this](const Expression* e) {
53.               out << ' ' << ActualType::Sign << ' ' << *e; } );
54.     return out << ')';
55.   }
56. };
57.
58. class Number: public Expression {
59.   double value;
60. public:
61.   Number(double d) : value(d) {}
62.   // Number* clone() const & override { return new Number(value); }
63.   Number* clone() && override { return new Number(value); }
64.   double eval() const override { return value; }
65.   ostream& print(ostream& out) const override {
66.     return out << value;
67.   }
68. };
69.
70. class Sum: public MultiExpression<Sum> {
71.   double evalImpl(double d1, double d2) const override {
72.     return d1 + d2;
73.   }
74. public:
75.   static constexpr char Sign = '+';
76.   using MultiExpression::MultiExpression;
77. };
78.
79. int main() {
80.     auto e1 = Sum( Number(2), Number(5) );
81.     auto e2 = Sum( std::move(e1), Sum( Number(1), Number(2) ) );
82.     std::cout << e2 << " = " << e2.eval() << std::endl;
83.     // std::cout << e1.eval() << std::endl;
84.     // auto e3 = Sum( e2, e2 );
85.     // std::cout << e3.eval() << std::endl;
86.     // e1 = Sum( Number(9), Number(9) );
87.     // std::cout << e1.eval() << std::endl;
88.     // e1 = Sum( Number(9), Number(9), Number(9) );
89.     // std::cout << e1.eval() << std::endl;
90. }
```

* במבחן עצמו נמסר תיקון ולפיו שורה 62 בהערה.

בהצלחה !